

Apache Pekko Introduction

Concurrent and Distributed Programming Course

Nicolas Farabegoli nicolas.farabegoli@unibo.it

10 May 2026

Department of Computer Science and Engineering (DISI) — University of Bologna

Today in one pass

- Why the actor model still matters for concurrent software
- What Apache Pekko provides on the JVM
- Typed actors, lifecycle, supervision, and interaction patterns
- Practical techniques: stash, FSMs, blocking isolation, discovery
- Testing actor behavior in isolation and in a real actor system
- Common mistakes, migration notes, and where to go next

Introduction

Typical distributed system challenges

! Components crash

Failures are the norm, not the exception.

! Messages get lost

Messages can be **lost**, **delayed**, or **duplicated**.

! Network instability

Latency, partitions, and **topology changes**.

Typical distributed system challenges

! Components crash

Failures are the norm, not the exception.

! Messages get lost

Messages can be **lost**, **delayed**, or **duplicated**.

! Network instability

Latency, partitions, and **topology changes**.

Designing with those in mind

These problems occur regularly in distributed systems, so we need to design with them in mind.

Why actors?

Easier concurrency

The actor model provides a level of abstraction that makes it **easier** to write correct **concurrent and distributed** software.

Asynchronous messages

Actors communicate **through asynchronous** messages, which **decouples** the sender and receiver and allows for more **flexible** and **scalable** designs.

Explicit protocols

The actor model encourages the use of **explicit protocols**, which can improve the clarity and maintainability of the code.

Toolkit overview

Apache Pekko is an open-source toolkit for building **concurrent, distributed**, and **resilient** message-driven applications on the JVM.

It carries forward the **actor-based** programming model, offers typed and classic APIs, and integrates modules for clustering, streams, persistence, and testing.

i Historical note

Many older slides, blog posts, and repositories still say *Akka*. When migrating code, the main change is usually from `akka.* / com.typesafe.akka` to `org.apache.pekko / pekko.*`, plus updated dependency coordinates.



Website: pekko.apache.org/

Documentation: pekko.apache.org/docs/pekko/current/

i Pekko API and DSL

Pekko provides APIs for developing actor-based systems with **Java** and **Scala** DSLs.

[Pekko Typed](#) new and type-safe API

[Pekko Classic](#) legacy API, still supported but not recommended for new code

Core modules

```
1 "org.apache.pekko" %% "pekko-actor-typed" % PekkoVersion
```

Provides basic support for typed actors and actor systems.

```
1 "org.apache.pekko" %% "pekko-stream" % PekkoVersion
```

Streams provide a higher-level abstraction on top of actors that simplifies writing such processing networks, handling all the fine details in the background and providing a safe, typed, composable programming model.

```
1 "org.apache.pekko" %% "pekko-remote" % PekkoVersion
```

Remoting enables actors that live on different computers to seamlessly exchange messages. While distributed as a JAR artifact, Remoting resembles a module more than it does a library.

Cluster and advanced modules

```
1 "org.apache.pekko" %% "pekko-cluster-typed" % PekkoVersion
```

Clustering gives you the ability to organize these into a “meta-system” tied together by a membership protocol.

```
1 "org.apache.pekko" %% "pekko-cluster-sharding-typed" % PekkoVersion
```

Sharding is a pattern that is mostly used together with Persistence to balance a large set of persistent entities.

```
1 "org.apache.pekko" %% "pekko-cluster-singleton" % PekkoVersion
```

Provides a mechanism to ensure only a single instance of a given actor runs across the cluster. While it introduces a common bottleneck, there are scenarios where its use is unavoidable.

```
1 "org.apache.pekko" %% "pekko-persistence-typed" % PekkoVersion
```

Persistence provides patterns to enable actors to persist events that lead to their current state.

Actor Foundations

Actors are entities that **encapsulate** state and behavior and interact solely through **asynchronous message passing**.

Behavior upon message arrival, an actor can:

- Send a finite number of messages to other actors
- Create a finite number of child actors
- (must) Specify the next behavior to handle the next message

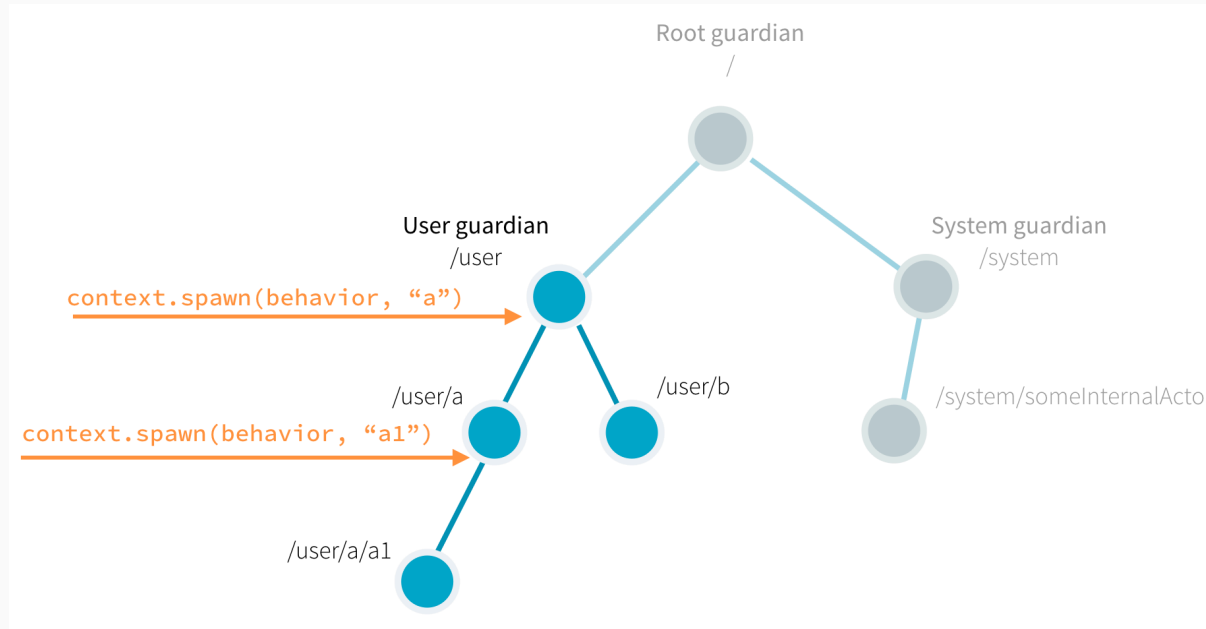
Encapsulation An actor is exposed to the outside through the `ActorRef[T]`

Actors are **logical** entities, decoupled by physical concurrency. An actor is a lightweight object (~300 bytes) that can be created in large numbers, and the runtime schedules them on a pool of threads.

Actor Architecture

An Actor in Pekko always belongs to a **parent**.

Actors are created by `ActorContext.spawn()` and the creator is the **parent** of the new actor.



- `/` is the **root guardian**, which supervises the system guardian and the user guardian.
- `/system` is the **system guardian**, which supervises system actors created by Pekko modules.
- `/user` is the **user guardian**, which supervises all actors created by user code.

Actor Architecture

Printing the `ActorRef` shows the hierarchy path.

basics / io.github.nicolasfara.es00.ActorHierarchyExample.scala

```
1  class PrintMyActorRef(context: ActorContext[String]) extends
2    AbstractBehavior[String](context) {
3
4    override def onMessage(msg: String): Behavior[String] =
5      msg match {
6        case "printit" =>
7          val secondRef = context.spawn(Behaviors.empty[String], "second-actor")
8          println(s"Second: $secondRef")
9          this
10     }
11 }
```

An **Actor System** is a hierarchical group of actors which share common configuration, including **dispatchers**, **mailboxes**, and **addresses**. It is the root of the actor hierarchy and provides the main entry point for creating actors.

! Heavyweight structure

It will allocate and manage **1..N threads**, and it is recommended to have one actor system per logical application boundary.

Hierarchical structure

- Splitting problems into smaller pieces
- Handling failures
- **Supervision model** (inspired by Erlang's “let it crash” philosophy)

Three ideas to keep

- `ActorSystem[T]` is the runtime root: create it once per logical application boundary.
- Actors live in a **hierarchy**, with user actors supervised by the user guardian.
- `ActorRef[T]` is the typed capability you share to let others send protocol messages.

i Path intuition

- An actor path names a position in the hierarchy, even when no actor is currently running there.
- A reference points to a concrete live actor incarnation at that path.
- Ordering is local: guaranteed for one sender and one receiver pair, not across the whole system.

Typed API

Getting started with Pekko Typed

```
1  val scala3Version = "3.7.4"
2  val PekkoVersion = "1.4.0"
3
4  libraryDependencies += Seq(
5    "org.apache.pekko" %% "pekko-actor-typed" % PekkoVersion,
6    "com.typesafe.scala-logging" %% "scala-logging" % "3.9.6",
7    "ch.qos.logback" % "logback-classic" % "1.5.32",
8    "org.apache.pekko" %% "pekko-actor-testkit-typed" % PekkoVersion % Test,
9    "org.scalatest" %% "scalatest" % "3.2.20" % Test,
10 )
```

Code repository with examples:

 <https://github.com/nicolasfara/seminar-pcd-actor-pekko-code>

Core typed abstractions

Behavior[T]

Describes how an actor handles one message of type τ .

ActorRef[T]

Typed capability for sending messages in protocol τ .

ActorSystem[T]

Runtime root and entry point of the application.

ActorContext[T]

Logging, spawning, adapters, scheduling hooks, and services.

```
1 object Counter:
2   enum Command:
3     case Tick
4
5   def apply(current: Int): Behavior[Command] =
6     Behaviors.receiveMessage:
7       case Command.Tick => Counter(current + 1)
```

Why it matters

- A Behavior[T] is the definition of how one actor reacts to messages of type τ .
- Returning a new behavior is how we model state changes in the functional style.
- Behaviors.same, Behaviors.stopped, and recursive behaviors are the common building blocks.

ActorRef[T]: the communication capability

```
1 object Counter:
2   enum Command:
3     case Tick
4     case Get(replyTo: ActorRef[Int])
5
6 // somewhere else
7 counterRef ! Counter.Command.Tick
8 counterRef ! Counter.Command.Get(replyTo = probe.ref)
```

i What to remember

- An `ActorRef[T]` is the only public handle you need to interact with an actor.
- The type parameter is the protocol: if a message is not in τ , it cannot be sent.
- Passing references around is how actors discover collaborators and where to send replies.

```
1 @main def runCounter(): Unit =  
2   val system = ActorSystem(Counter(0), "counter-system")  
3   system ! Counter.Command.Tick
```

Operational role

- ActorSystem[T] starts the top-level behavior and owns the runtime infrastructure.
- It manages dispatchers, scheduling, configuration, and the root guardians of the hierarchy.
- In practice, we usually create one actor system per application or bounded subsystem.

```
1 def apply(): Behavior[Command] = Behaviors.setup: context =>
2   val worker = context.spawn(Worker(), "worker")
3   context.log.info("Worker started: {}", worker)
4   Behaviors.receiveMessage:
5     case Start =>
6       worker ! Worker.Command.Run
7       Behaviors.same
```

Common uses

- ActorContext[T] gives access to services that only make sense from inside an actor.
- Typical operations are `spawn`, `watch`, logging, adapters, and scheduling-related helpers.
- Keep the context local to the actor: do not treat it as a dependency to pass everywhere.

Actor as a function

An **actor** can be expressed as a **function** that takes the current state and returns a behavior for the next message. This is the most common style in Pekko Typed, and it fits well with immutable state and finite-state machines.

```
1 object CounterActor:
2   enum Cmd:
3     case Tick
4
5   def apply(from: Int, to: Int): Behavior[Cmd] = Behaviors.receive: (_, message) =>
6     message match
7       case Tick if from < to => CounterActor(from + 1, to)
8       case _ => Behaviors.stopped
```


Actor as an object

For actors with complex mutable state, it can be more readable to use an object-oriented style and keep the state in mutable fields. This is a valid choice as long as the **mutable state is properly encapsulated** and **not shared** outside the actor.

```
1 class Counter(context: ActorContext[Command], var from: Int, val to: Int) extends
2   AbstractBehavior[Command](context):
3   def onMessage(msg: Command): Behavior[Command] = msg match
4     case Command.Tick if from < to =>
5       context.log.info(s"Counter: $from")
6       from += 1
7       this
8     case _ => Behaviors.stopped
```

Actor Lifecycle

Actor lifecycle basics

- Actors are created when spawned: `context.spawn(behavior, "name")`
- Actors are stopped when they return `Behaviors.stopped`
- When a parent stops, **all children are recursively stopped** before the parent's `PostStop` signal

! Resource cleanup

This hierarchical stopping behavior **greatly simplifies resource cleanup** and helps avoid resource leaks from open sockets, files, and other concurrent resources.

The root guardian

Create the **root user guardian** with `ActorSystem`(behavior, "name"):

```
1 ActorSystem(Behaviors.setup(new MyRoot(_)), "mysystem")
```

Child actors

Create child actors with `ActorContext[T].spawn`(behavior, "name"):

```
1 Behaviors.setup { context =>
2   val child = context.spawn(someBehavior(), "child")
3   context.log.info(s"Child created: $child")
4   Behaviors.empty[String]
5 }
```

Self-stopping

Return `Behaviors.stopped` when the actor is done or in response to a user-defined stop message. This is the recommended pattern.

Stopping children

Call `context.stop(childRef)` from the parent. You cannot stop arbitrary actors this way.

Lifecycle signals: PostStop

PostStop signal

Sent just after the actor has been stopped. No messages are processed after this point, but cleanup logic can be run here.

```
1  override def onSignal: PartialFunction[Signal, Behavior[String]] =  
2    case PostStop =>  
3      context.log.info("Actor stopped, cleanup running")  
4    this
```

i Strict ordering

All `PostStop` signals of children are processed **before** the `PostStop` signal of the parent.

Lifecycle example: parent and child

basics / io.github.nicolasfara.es00.basics.ActorLifecycleExample.scala

```
1 object StartStopActor:
2   def apply(): Behavior[String] = Behaviors.setup: context =>
3     context.log.info(s"Actor ${context.self} started")
4     val _ = context.spawn(ChildActor(), "child-actor")
5     Behaviors.receiveMessage[String]:
6       case "stop" => Behaviors.stopped
7       case _      => Behaviors.same
8   object ChildActor:
9     def apply(): Behavior[String] = Behaviors.setup: context =>
10       context.log.info(s"Actor ${context.self} started")
11       Behaviors.receiveSignal:
12         case (ctx, PostStop) =>
13           context.log.info(s"Actor ${ctx.self} stopping")
14           Behaviors.same
```

Lifecycle execution order

When we send "stop" to the first actor:

```
1 first started    // first actor created
2 second started   // child spawned
3 second stopped   // child stops first
4 first stopped    // then parent stops
```

Key insight

The **strict parent-child ordering** ensures that children always clean up before their parents, preventing orphaned resources and simplifying resource management.

Validation error vs failure

Validation error should be modeled as part of the protocol

Failure an unexpected exception that should be handled by the supervision strategy.

“Let it crash” is the recommended approach: if an actor fails, it should be stopped and restarted in a clean state by **its parent**.

! Default failure strategy

Actors are **stopped by default** if an exception is thrown and no supervision strategy is defined.

What it is

Supervision lets a parent **declaratively define** what to do when child actors fail with specific exceptions.

Where to apply

Wrap child behavior with `Behaviors.supervise(...)`, usually where the parent spawns the child.

Common supervision strategies

```
1 Behaviors.supervise(behavior)
2   .onFailure[IllegalStateException](SupervisorStrategy.restart)
3 Behaviors.supervise(behavior)
4   .onFailure[IllegalStateException](SupervisorStrategy.resume)
5 Behaviors.supervise(behavior)
6   .onFailure[IllegalStateException](
7     SupervisorStrategy.restart.withLimit(
8       maxNrOfRetries = 10,
9       withinTimeRange = 10.seconds,
10    )
11  )
12 Behaviors.supervise(behavior)
13   .onFailure[IllegalStateException](SupervisorStrategy.restart)
14   .onFailure[IllegalArgumentException](SupervisorStrategy.stop)
```

Restart semantics and wrapping

! Important note

On restart, Pekko re-installs the **original behavior** passed to `Behaviors.supervise`. If mutable state is involved, create actors through `Behaviors.setup`.

```
1 def apply(): Behavior[Command] =  
2   Behaviors.supervise(counter(1)).onFailure(SupervisorStrategy.restart)
```

i Wrapping behavior

In functional style, applying supervision at the top-level is enough: returned behaviors are re-wrapped automatically.

Escalating failures

Given the parent-child hierarchy, if a child fails, the parent can **get notified**.

basics / io.github.nicolasfara.es00.basics.ParentFailureNotification.scala

```
1  object ParentActor:
2    def apply(): Behavior[String] = Behaviors.setup: ctx =>
3      ctx.watch(ctx.spawn(ChildFailingActor(), "child-actor"))
4      Behaviors.receiveSignal:
5        case (_, Terminated(ref)) =>
6          ctx.log.info(s"Child $ref has terminated")
7          Behaviors.stopped
8
9  object ChildFailingActor:
10    def apply(): Behavior[String] = Behaviors.setup: _ =>
11      throw new RuntimeException("I failed")
```

Basic Techniques

Fire and forget

Pekko's core API is designed for **fire-and-forget** message sending, which is the most common interaction pattern. You can send messages to an actor without expecting a reply, and the actor will process them **asynchronously**.

- Pekko delivery guarantee: **at-most-once** delivery, which means that messages may be lost but will not be duplicated.
- Pekko ordering guarantee: message ordering is guaranteed **per sender-receiver** pair, but not across the whole system.

Request-response

For interactions that require a reply, the common pattern is to include an `ActorRef` in the message for the reply to be sent to.

```
1 enum Protocol:  
2   case Request(data: String, replyTo: ActorRef[Response])  
3   case Response(result: String)
```

This way, the sender can provide a reference for where the response should be sent, and the receiver can reply directly to that reference.

Request-response with ask

basics / io.github.nicolasfara.es00.basics.AskOperator.scala

```
1 given Timeout = 3.seconds
2
3 val greeting: Future[GreeterActor.Greeting] = target ? { replyTo =>
4   GreeterActor.Command.Greet("Pekko", replyTo)
5 }
```

When `ask` is useful

- Bridge actor messages to `Future`-based APIs.
- Request one reply with a clear timeout boundary.
- Keep protocol explicit: the request still carries a typed `replyTo`.

Stashing enables an actor to **temporarily buffer messages** that cannot be processed in the current state, and then process them later when the state changes.

- If the actor needs to initialize some state or create some resources before it can handle certain messages, it can stash those messages until it's ready.
- When the actor needs to wait for an external result (e.g., a response from another actor or a future), it can stash incoming messages until the result is available.

Stash

Buffer messages that cannot yet be processed, for example while loading initial state or waiting for an external result.

Finite-state logic

Model each state as a behavior. The behavior returned after one message encodes the next state.

basics / io.github.nicolasfara.es00.basics.Stashing.scala

```
1 object Stashing:
2   // Command protocol ...
3   def apply(id: UUID, db: DB): Behavior[Command] = Behaviors.withStash(100): buff =>
4     Behaviors.setup(context => new Stashing(context, buff, id, db).start())
```

Testing

Testing typed actors

Prefer `BehaviorTestKit` **when**

- The behavior itself is the unit under test
- You want deterministic, synchronous execution
- You need to inspect effects such as spawn, watch, and logs

Prefer `ActorTestKit` **when**

- Actors collaborate inside a real actor system
- Timing, scheduling, or protocol wiring matter
- Test probes should observe replies like controlled mailboxes

i Rule of thumb

Start with `BehaviorTestKit`; move to `ActorTestKit` when interaction between actors is the real subject of the test.

Synchronous test with BehaviorTestKit

```
1 import org.apache.pekko.actor.testkit.typed.scaladsl.{BehaviorTestKit, TestInbox}
2
3 val testKit = BehaviorTestKit(Counter(0))
4 val inbox = TestInbox[Int]()
5
6 testKit.run(Counter.Command.Tick)
7 testKit.run(Counter.Command.Get(inbox.ref))
8 inbox.expectMessage(1)
```

Why this style is useful

- No scheduler races or timing concerns
- Outgoing messages are easy to inspect
- State transitions and protocol logic stay cheap to test

Asynchronous test with ActorTestKit

```
1  import org.apache.pekko.actor.testkit.typed.scaladsl.ActorTestKit
2  import org.scalatest.wordspec.AnyWordSpecLike
3
4  class CounterSpec extends AnyWordSpecLike:
5    val testKit = ActorTestKit()
6
7    "Counter" should {
8      "reply with the current value" in {
9        val counter = testKit.spawn(Counter(2))
10       val probe = testKit.createTestProbe[Int]()
11       counter ! Counter.Command.Get(probe.ref)
12       probe.expectMessage(2)
13     }
14   }
```

Why probes matter

- A probe is a mailbox you can assert against
- Ideal for reply-oriented protocols
- Useful when behavior depends on real actor interaction

Wrap-up

! Frequent pitfalls

- Blocking directly in handlers
- Using ask everywhere instead of message-driven protocols
- Treating supervision as input validation
- Exposing internal implementation details in the public protocol
- Building giant actors instead of small collaborators with clear ownership

What we learned and what comes next

Key takeaways

- Actors provide isolation, explicit protocols, and structured failure handling.
- Pekko Typed makes message contracts first-class through types.
- Lifecycle, supervision, timers, stash, and testing are the practical core.
- The same foundations extend to remoting, clustering, persistence, and streams.

i Suggested follow-up topics

Next lectures can build on this deck with cluster membership, sharding, persistence, and stream integration.

Acknowledgement

The original seminar structure and part of the teaching material derive from earlier Akka-oriented slides used in the course and from Roberto Casadei's material.

Primary references

- Apache Pekko documentation: pekko.apache.org/docs/pekko/current/
- Apache Pekko modules overview: pekko.apache.org/modules.html
- Typed actor discovery and dependency snippets: typed/actor-discovery
- Typed testing guide: typed/testing